2025 IEEE International Conference on Big Data and Smart Computing (BigComp) | 979-8-3315-2902-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/BigComp64353.2025.00062

InsightCube: Accelerating Interactive Insight Discovery in Exploratory Data Analysis

Hanbing Zhang[1], Yinan Jing[1,2,3,*], Wei Zhang[1], Zhenying He[1,2], Kai Zhang[1,2], X. Sean Wang[1,2],
Zhenqiang Chen[4], Changchun Zhang[4]
{hbzhang17, jingyn,wei_zhang21, zhenying, zhangk, xywangcs}@fudan.edu.cn,
{zhenqiang.chen, changchun.zhang}@transwarp.io
[1] School of Computer Science, Fudan University, Shanghai, China
[2] Shanghai Key Laboratory of Data Science, Shanghai, China
[3]Shanghai Key Laboratory of Computer Software Evaluating and Testing, Shanghai, China
[4]Transwarp Technology(Shanghai) Co., Ltd., Shanghai, China

***Abstract*—Exploratory data analysis (EDA) is usually human-in-the-loop and time tedious since identifying insights that show interesting characteristics and trends in data is laborious. Insight discovery helps users gain structured knowledge when understanding the dataset by categorizing interesting characteristics and trends into commonnesses and exceptions. However, existing insight discovery methods cannot interact with users when discovering insights. More importantly, the efficiency of insight discovery for these methods is low. In this paper, we propose a framework that supports and accelerates interactive insight discovery. We first divide the dataset into multiple subsets, and precompute the pattern snippets that include the distribution characteristics of subsets. Considering efficiency, we only precompute for subsets that cannot be further split. Then we design a novel cube structure, called InsightCube, to store precomputed pattern snippets. In order to accelerate insight discovery, we reuse the precomputed pattern snippets stored in InsightCube to infer the pattern of subsets that cannot be split. Furthermore, during insight discovery, we propose pruning strategies to accelerate the pattern inference for subsets that can be split. Experimental results on real-world datasets show that InsightCube improves the insight discovery efficiency by up to 2.5× compared to the state-of-the-art insight discovery method.**

***Index Terms*—exploratory data analysis, interactive insight discovery, data cube, efficiency optimization**

I. Introduction

As the first step in data analysis, Exploratory Data Analysis (EDA) is used to understand the characteristic information of the dataset and gain knowledge from the dataset [20]. Traditional EDA methods [13], [17] gain knowledge by executing SQL to fetch subsets of dataset and extracting the statistical information and distribution trends of subsets. However, the knowledge gained from a single query contains limited information. The information associated with different queries is disjointed, making it difficult to further generalize to gain more information [14].

Insight discovery [1], [6], [14], [19] is an emerging topic to gain structured knowledge in the EDA. These methods not only extract basic patterns from a single query, but generalize patterns from multiple queries or queries executed in multiple steps. Patterns of queries form structured knowledge, called *insight*, that encompasses more information. However, these methods may not fully meet the analytical needs of users.

We use an example to illustrate the user's insight discovery process. As shown in Fig. 1, users want to analyze a supermarket dataset. Firstly, as shown ① in Fig.1, they are interested in the subset of the segment of "Home Office". They write multiple queries and discover an insight from the results: most regions have the largest $MAX(discount)$ on the category of "Office Supplies". The distribution of $MAX(discount)$ in the west region is an exception because the values are the same in all categories. Then, they proceed to write queries to gain insights from other data in the subset. This process takes T_1 seconds. Next, as ② and ③ in Fig.1, they are interested in the segment of "Corporate" and the total data of segment of ["Office Supplies", "Corporate"], and perform explorations similar to ① to discover insights. These processes of discovering insights from subsets take T_2 seconds and T_3 seconds, respectively. Finally, as ④ in Fig.1, users continue to discover insights in subsets in which they are interested.

From the above example of insight discovery, we find that insights can be discovered from multiple queries executed by the user, and users may select the subset that they are interested in exploring. In this scenario, insight discovery methods enable to help users divide and analyze subsets to see whether insights exist. But they face some challenges. **Challenge 1.** Many existing insight discovery methods only support to discover insights end-to-end. Users cannot interact with the system. The system doesn't support users to filter the data they are interested in or perform further analysis on the insights they have discovered. **Challenge 2.** During the iterative and interactive process of insight discovery by users, insight discovery methods have the problem of redundant calculations. As shown in Fig.1, the analysis operations of ① and ② respectively analyze the subset with segments of

* Yinan Jing is the corresponding author.

2375-9356/25/$31.00 ©2025 IEEE
DOI 10.1109/BigComp64353.2025.00062

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

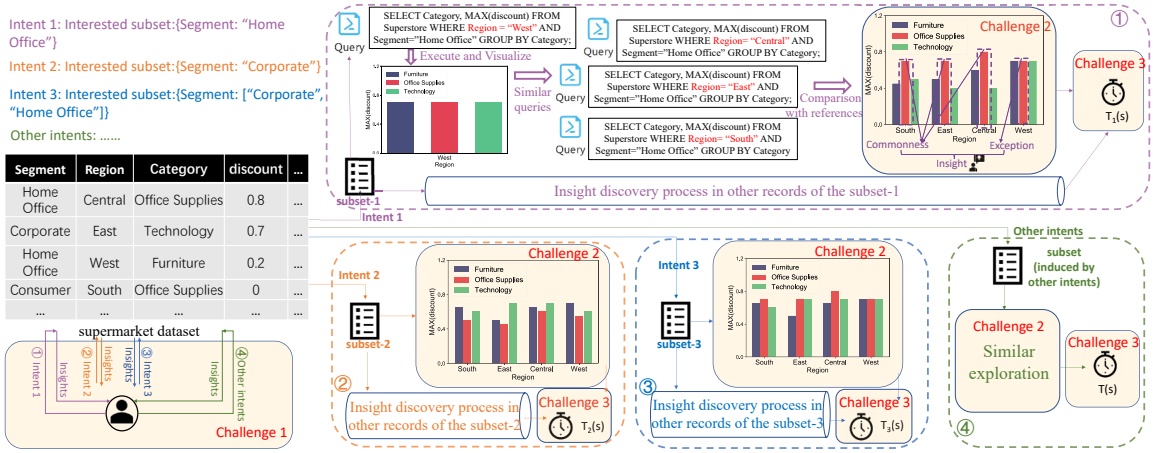


Fig. 1. Example of insight discovery during EDA

"Home Office" and "Corporate". In the analysis process of ③, where the segment is ["Home Office", "Corporate"], existing insight discovery methods cannot reuse the calculation results in ① and ②, so insights need to be recalculated with the great cost. **Challenge 3.** The time overhead of discovering insights through existing insight discovery methods is too high. In the scenario of interactive data analysis, it is difficult to complete the insight discovery process within an acceptable time budget.

In this paper, we propose an interactive insight discovery acceleration framework, which enables the insight discovery to interact with users (**Challenge 1**). In order to solve the problem of redundant calculations, we propose InsightCube, a cube structure extended by the datacube [10]. We precompute the distribution characteristics of subsets, called pattern snippets, and store them in InsightCube. Considering the time and space cost of pre-computation, we only precompute for subsets that cannot be split (**Challenge 2**). Finally, in order to enhance the efficiency of insight discovery, we reuse the precomputed pattern snippets stored in InsightCube to infer the pattern of subsets that cannot be split. As for subsets that can be split, it is proposed to accelerate the pattern inference process with InsightCube and pruning strategies (**Challenge 3**).

In summary, the contributions of this paper are as follows:

- We propose a framework that supports interactive insight discovery. Furthermore, the framework accelerates the discovery process.
- We propose a series of methods to accelerate the interactive insight discovery. In the offline stage, we propose to build InsightCube to store the statistical information and distribution characteristics of subsets that cannot be split, which will be reused. In the online stage, we propose pruning strategies to accelerate the pattern inference of unknown subsets.
- We extend the existing insight discovery method to support interaction. We compare the efficiency of InsightCube and other methods on real-world datasets. Experimental results show that InsightCube improves efficiency by up to 2.5× compared to the state-of-the-art insight discovery method.

II. Problem Formulation & System Architecture

A. Preliminaries

The object of EDA is often tabular data. We use $\mathbb{D} = \langle Dim, M \rangle$ to represent a data table, where $Dim = \{dim_1, \ldots, dim_d\}$ is the set of dimensional attributes in the table, $M = \{m_1, \ldots, m_m\}$ is the set of measurement attributes in the table. In the EDA process in which users participate, users usually continuously execute queries to obtain the specific subset in the table, and further analyze its distribution characteristics in the next step. The subset can be obtained by executing a query on the dataset, and we define the query executed by the user to represent this subset as the data scope (Definition 1). The characteristic of the distribution of the subset is defined as the pattern snippet (Definition 2).

Definition 1 (Data Scope): A data scope (ds) is a 3-tuple that consists of subspace s, breakdown b, and measure m, i.e., $ds = \langle s, b, m \rangle$, where $subspace = \{s_1, \ldots, s_{|d|}\}$is a set of filters on d dimensions. b is the dimension where a group-by operator is performed. m is the attribute performed by an aggregation.

Definition 2 (Pattern Snippet): For a given data scope, the distribution characteristic of ds is defined as a pattern snippet ps that is a 3-tuple. $ps = \langle ds, t, h \rangle$, where t is the pattern type and h is the highlight in the group of ds.

The distribution characteristics of the data scope can be represented by the characteristics of a single point as well as the distribution of all points. In InsightCube, we implement 8 pattern types that are commonly used in EDA and existing methods [6], [14]. The pattern snippet can be inferred by calling pattern analysis programs of these eight pattern types.

A single pattern snippet contains a little information and are not convincing enough. Therefore, we extend the data scope by sibling groups, so that the patterns of multiple data scopes can refer to each other. For a given subspace s, we define the sibling group of the subspace s on the i-th dimension attribute as $SG(s, i) = \{s' | \forall j \neq i, s'_j = s_j, s'_i \neq$

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

$\{*\}\}$. In addition, we define the extending operation as $Exd_{s_i}(\langle s, b, m\rangle) = \{\langle s^{'}, b, m\rangle | s^{'} \in SG(s, i)\}$. We define a set of expanded data scopes as the homogeneous data scope hds (Definition 3), whose pattern snippets can refer to each other. The premise that pattern snippets can refer to each other is that the pattern types are the same. Therefore, we define patterns and highlights that can be used for reference in multiple data scopes as the homogeneous pattern snippet hps (Definition 4), which must satisfy all data scopes and have this pattern type at the same time.

Definition 3 (Homogeneous Data Scope): The homogeneous data scope hds derived from ds is a set of data scopes that are extended by $Exd(ds)$, i.e., $hds(ds_i) = \{ds | ds \in Exd(ds_i)\}$.

Definition 4 (Homogeneous Pattern Snippet): The homogeneous pattern snippet hps_t is a set of type-induced pattern snippets derived from hds, i.e., $hps_t(hds) = \{ps(type, ds) | \forall\ ds \in hds, evaluate(ds, type) = true\}$.

We next analyze the similarities and differences of the highlight of the patterns. We define that only two pattern snippets with the same pattern type and highlights are considered similar, i.e., $Sim(ps_i, ps_j) = True$, if and only if $ps_i.t = ps_j.t$ and $ps_i.h = ps_j.h$. Given a homogeneous pattern snippet, divide the homogeneous data scope into multiple similar sets according to the highlights of each pattern type. If there is a highlight set in which the number of data scopes exceeds the threshold τ, this highlight is defined as the commonness of the homogeneous pattern snippet in this pattern type. Commonnesses of all pattern types constitute a commonness set, defined as $CommonSet = \{Com | Com \in hps/Sim\ s.t. \frac{|Com|}{|hps|} > \tau\}$, where / represent the quotient set. In addition to commonness, we define other highlights that do not belong to the commonness set as exceptions. Exceptions of all pattern snippets constitute the exception set $Exception$ (Exc). The definition of insight is based on them.

Definition 5 (Insight): If $CommonSet \neq \emptyset$, the insight $Insight(hps) = \langle CommonSet, Exc\rangle$ indicates that insights consist of $CommonSet$ and Exc among homogeneous pattern snippets. We quantify the value of insight by follows:

$$Score(Insight) = 1 - \frac{S + reg}{S^*} \tag{1}$$

, where $S = -\sum_{i=1}^{n} \frac{|Com_i|}{|hps|} \log \frac{|Com_i|}{|hps|}$ is an entropy-like value, n is the size of $|CommonSet|$, S^* is the upper bound of S and reg is a regularization term to penalize the case that all data scopes have the same highlight of pattern snippet.

B. Interactive Insight Discovery

The process of insight discovery in EDA is as follows. Firstly, the system generates a data scope list $DSList_{All} = \{ds_0, ds_1, \ldots, ds_n\}$ based on dataset's metadata. Secondly, the system expands each data scope, whose pattern snippet is not null, into the homogeneous data scope $hds(ds_i)$ to obtain a list of homogeneous data scopes $HDSList_{All} = \{hds_0, hds_1, \ldots, hds_m\}$. Thirdly, the system calculates the homogeneous pattern snippet of each hds. Then system calculates the insight list $InsL_{All} = \{insight_0, insight_1, \ldots, insight_m\}$. The purpose of insight discovery is to recommend top-k insight items with the highest cumulative benefits to users. We define the cumulative benefits of an insight list containing k insights from the perspective of scoring and diversity as follows:

$$P(InsL) = R(InsL) * \sum_{is \in InsL} Score(is) \tag{2}$$

$$R(InsL) = \frac{\alpha * |InsL_s| + \beta * |InsL_b| + \gamma * |InsL_m|}{(\alpha + \beta + \gamma) * |InsL|} \tag{3}$$

, where R is used to describe the diversity of insights, among which $|InsL_s|$, $|InsL_b|$ and $|InsL_m|$ respectively correspond to the unique number of subspaces, breakdowns, and measures. α, β and γ are weight coefficients, and the default setting are [1,1,1]. Therefore, if the data scope list corresponding to the recommended insight list contains the same tuple items, $R(InsL) < 1$.

The insight discovery described above defaults to sufficient time to calculate the dataset for insight discovery. However, in actual exploratory analysis scenarios, users cannot tolerate the long waiting time for calculations. Users may be interested in only part of the data or part of the analysis results. Therefore, interactive insight discovery based on the intention of the user to interact is defined as $\langle Intent\rangle \rightarrow \langle TimeBudget\rangle^* | \langle Operation\rangle^*$.

The user's interaction intention can be composed of the maximum waiting time ($TimeBudget$) or the interaction action ($Operation$). There are two types of user interaction actions: one is to select the subset of interest ($filter$). The other is that after seeing the insights, users can perform further drill-down analysis ($drilldown$) on the individual insight displayed. We expand the definition of data scope to respond to the user's intention and define the expanded data scope as $ds = \langle subspace, breakdown, measure, operate\rangle$, where $operate = filter | drilldown$. Finally, based on the user-given or default time budget, the interactive insight discovery is calculated as follows:

$$\begin{aligned} Rec(InsL) = & \underset{InsL=\{is_1,\ldots,ist_k \subset InsL_{All}\}}{\arg\max} P(InsL) \\ & s.t. TC(HDSList) \leq TimeBudget \end{aligned} \tag{4}$$

, where $TC(\cdot)$ represents the calculation time of $HDSList$, and $InsL$ is calculated based on $HDSList$.

C. System Architecture

In this paper, we propose an interactive insight discovery acceleration framework (as shown in Fig. 2). It divides interactive insight discovery into two stages. In the offline stage, the system infers the pattern snippets of data scopes and stores them in the InsightCube. In the online stage, the system adopts the InsightCube and pruning strategies to accelerate the pattern inference of data scopes, thus achieving the purpose of accelerating insight discovery.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

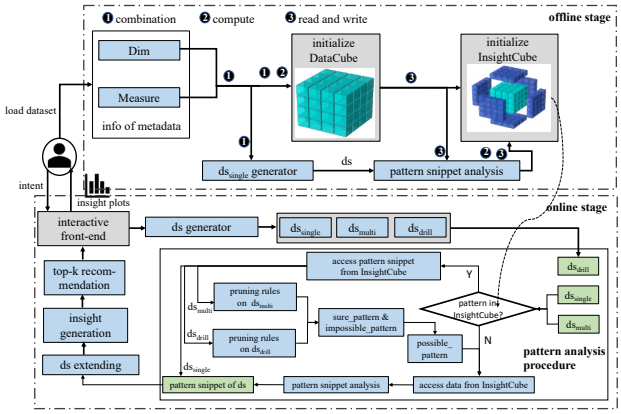


Fig. 2. Framework of interactive insight discovery with InsightCube

Example	Data Scope				Data
	subspace	breakdown	measure	subset	
ds_1	Category: Furniture	Region	MAX(discount)	Segment: Home Office	[0.6, 0.5, 0.45, **0.5**]
ds_2	Region: West	Category	MAX(discount)	Segment: Home Office	[**0.5**, 0.7, 0.7]
ds_3	Region: West	Category	MAX(discount)	Segment: Corporate	[0.7, 0.5, 0.7]
ds_4	Region: West	Category	MAX(discount)	Segment: [Home Office, Corporate]	[0.7, 0.7, 0.7]
ds_5	Segment: Home Office	Category	MAX(discount)	Region: West	[0.5, 0.7, 0.7]

Fig. 3. Examples of data scope

III. Method

A. Offline Construction of InsightCube

In order to quantitatively analyze the time cost in the offline stage, we assume the metadata of the dataset: suppose a dataset has m measure attributes and n dimensional attributes, where the cardinality of the dimensional attributes is k. There are a kinds of aggregations. It is not difficult to calculate the number of data scopes that are exponentially related to k and n. Precomputation of subset and their pattern snippets can only be done for part of subsets within an acceptable overhead. We analyze the data in the data scope, as shown in Fig. 3. Aggregating the data of ds_2 and ds_3 can obtain the data of ds_4. In addition, two data scopes may contain partially identical data (such as ds_1 and ds_2), and the same data may exist in two data scopes (such as ds_2 and ds_5).

Based on the above analysis, we first divide data scopes further. We define the data scope that satisfies each attribute to contain at most one value as the single-value data scope (ds_{single}, e.g., ds_1 in the Fig. 3), and the other data scope is the multi-value data scope (ds_{multi}, e.g., ds_4 in Fig. 3). In order to avoid redundant calculations, we refer to and expand the DataCube technology to precompute the data and patterns of ds_{single}. The expanded cube is called InsightCube, which contains the aggregation results in certain dimensions. A dataset has $a * m$ InsightCubes. An InsightCube consists of multiple $CuboidSet_i$, which contains all grouped sets containing i dimensional attributes in the dataset. The element of $CuboidSet_i$ is $Cuboid(DimSet_{ij})$, which represents the aggregation result of the $j-th$ combination among the combinations of i attributes and the pattern snippet of ds_{single}. The construction and usage of cells will be described below.

Construction and usage of data cells. The data cells in $Cuboid(DimSet_{ij})$ are defined as $cell_{data} = \langle a_1, a_2, \ldots, a_i : value \rangle$, where $a_q (1 \leq q \leq i)$ represents the specific value of the $q-th$ dimensional attribute in the Cuboid's attributes, and $value$ is the aggregation result of the measure in the Cuboid. An example of a three-dimensional Cuboid is shown in the green cells in the middle ($Part\ A$) of Fig. 4. The data in a single-value data scope corresponds to all cells on the $breakdown$ attribute in Cuboid. For example, the purple cells in the red circle correspond to the data of ds_2 and ds_5 shown in Fig. 3. The $Part\ B$ of the figure shows how the data cells complete the calculation of the data scopes shown in Fig. 3 by reusing data cells. Then, we execute SQL at least once to get a Cuboid result. The query groups Cuboid's attributes and performs aggregation operations on a measure attribute. Considering that the same $Cuboid(DimSet_{ij})$ in different InsightCubes has the same combination of dimensional attributes, we can merge all measures for the combination of dimensional attributes and calculate multiple Cuboids through the augmented query instead of calculating Cuboids one by one.

Construction and usage of pattern cells. Pattern cells in $Cuboid(DimSet_{ij})$ that store the pattern snippet of ds_{single} are defined as $cell_{pattern} = \langle a_1, a_2, \ldots, a_i : pattern_dict \rangle$. $a_q (1 \leq q \leq i)$ represents the specific value of the $q-th$ dimension attribute in the Cuboid's attributes, indicating the value of data scope's tuple items, of which there is and only one attribute whose value is "ALL", corresponding to the data scope's breakdown item. An example of pattern cells of a three-dimensional Cuboid is shown in the blue cells in Part A of Fig. 4. The pattern cell in the yellow circle is $cell_{pattern} = \langle West, ALL, Home\ Office : \{\text{“}trend\text{”} : \text{“}increase\text{”}, \text{“}outlier\text{”} : [\text{“}Furniture\text{”}]\} \rangle$, which indicates ds_2 shown in Fig. 3 is increasing and the category of "Furniture" is an outlier. The Part C of Fig. 4 shows the inference process of the pattern cell.

B. Online Interactive Insight Discovery with InsightCube

During the interactive insight discovery process, users may select subsets of interest to discover insight or drill down into discovered insights, repeating the process of insight discovery. The entire process of insight discovery is illustrated in the online stage of Fig. 2. Through preliminary experiments, we find that the most time-consuming stage in insight discovery is the calculation of ds. The key to accelerating the insight discovery process lies in accelerating the calculation of ds. It can be achieved by using InsightCube. The calculation of ds includes calculating its data and pattern. Depending on the user's intent, we can categorize ds into ds_{single}, ds_{multi} and ds_{drill} ('operate' item set to 'drill-down').

The data and pattern of ds_{single} can be obtained by accessing the InsightCube directly. The data of ds_{multi} can be obtained by aggregating multiple ds_{single}, but its pattern cannot be inferred by reusing pattern cells directly. However,

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

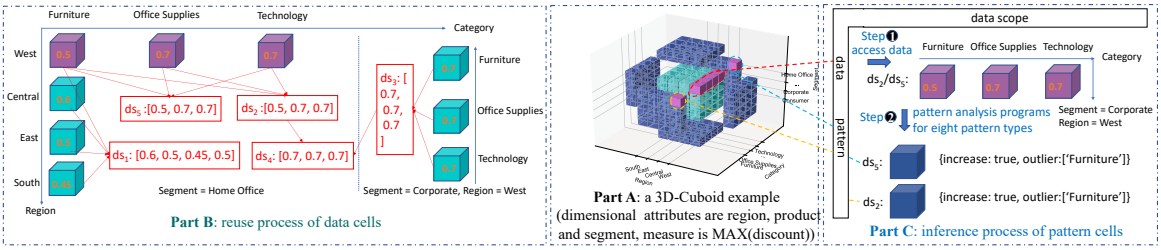


Fig. 4. Examples of cells in a three-dimensional (3D) Cuboid of InsightCube

multiple ds_{single} can be combined to obtain ds_{multi}. There is a certain correlation between the pattern snippet of ds_{multi} and multiple ds_{single} that makeup it. Therefore, we can infer a partial pattern snippet of ds_{multi} based on the pattern snippet of ds_{single}, thereby reducing the number of pattern types to infer.

Pattern snippets pruning for ds_{multi}. Based on the above analysis, we find that before executing the pattern analysis programs on ds_{multi}, some pattern types of it can be inferred based on the pattern snippets of the multiple ds_{single} that makeup it. Given two single-value data scopes ds_{single} and $ds^{'}_{single}$, a multi-value data scope ds_{multi} can be obtained by combining their data. We define the process of inferring the partial pattern snippet of ds_{multi} from ds_{single} and $ds^{'}_{single}$ as $Combine(ds_{single}.ps, ds^{'}_{single}.ps) = \{sure_ps, impossible_ps\}$, where $Combine(\cdot,\cdot)$ represents the constraints between the patterns before and after combination. $sure_ps$ and $impossible_ps$ respectively represent the pattern types and highlights that must exist and those that do not exist in ds_{multi}. Thus, for ds_{multi}, the potential pattern types that need to be inferred through the pattern analysis programs are $possible_ps = type \setminus (sure_ps \bigcup impossible_ps)$, where $\setminus$ is the difference set symbol.

By reusing the pattern cells of ds_{single}, the pattern inference process of ds_{multi} can be pruned. Since the combination process is related to the aggregation, we analyze the pruning rules in four aggregations, namely sum aggregation (SUM), maximum aggregation (MAX), average aggregation (AVG), and minimum aggregation (MIN). We analyze the pruning rules on ds_{multi} for aggregations and provide rigorous proofs for inferences.

Property 1 (Pruning Rules for Pattern Inference on ds_{multi}): We find a total of 10 rules that can be used for inferring. Take a rule in the sum aggregation as an example. The rule is: if $ds_1 = \{p_1, \ldots, p_n\}$ and $ds_2 = \{q_1, \ldots, q_n\}$ both have the "outstanding_1" pattern and the highlight of "outstanding_1" is the same, then the combined ds have "outstanding_1" pattern, and its highlight is the highlight of ds_1 (ds_2). This rule also applies to types of "outstanding_n" and "attribution".

Proof 1: Assuming that the means of ds_1 and ds_2 are a and b, then the data scope obtained with the sum aggregation is $ds = \{p_1 + q_1, \ldots, p_n + q_n\}$, the mean is $(a + b)$. Before the combination, ds_1 and ds_2 both have the "outstanding_1" type and the highlight of the two ds is the same. Assume that the highlight is the $i-th$ group, that is, $2 * a < p_i < (n * a)/2$, $2*b < q_i < (n*a)/2$, then it is easy to get $2*(a+b) < ds_i = p_i + q_i < (n*(a+b))/2$, and $a+b$ and $n*(a+b)$ are exactly the mean and sum of the data in ds, this formula satisfies the definition of the "outstanding_1" type. In the same way, it can be proved that the "outstanding_n" and "attribution" types also satisfy this rule.

We use o_1, o_l, a, cp, $outlier$, $trend$, e, and $season$ to sequentially represent the pattern types in [6], [14]. The mentioned rule can be succinctly represented as $(x, x) \rightarrow \{\{x\}, \{\}\}, x \in \{o_1, o_l, a\}$. Other rules in the SUM are $(e, trend) \rightarrow \{\{trend\}\{\}\}$; $(\neg x, \neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{o_1, o_l, a\}$. $\neg x$ represents the ds doesn't satisfy the x pattern type. Rules in the MAX are $(x, x) \rightarrow \{\{x\}, \{\}\}, x \in \{o_l, trend, e\}$; $(\neg x, \neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{o_1, a\}$. Rules in the MIN are $(o_1, o_1) \rightarrow \{\{\}, \{e, season\}\}$; $(x, x) \rightarrow \{\{x\}, \{\}\}, x \in \{a, trend, e\}$;$(\neg o_l, \neg o_l) \rightarrow \{\{\}, \{o_l\}\}$. There are two rules in the AVG. The first one is if ds_{multi} satisfy e, then it doesn't satisfy $o_1, o_l, a, cp, outlier, trend$. The second one is $trend$ and $season$, which will not be satisfied at the same time. o_1 and a are also like this.

Fig. 5 shows an example of a pattern inference process in the online stage using pruning rules to accelerate. The two ds_{single} before combination both have the pattern of "outstanding_1" with the highlight of Feb. Through pruning rules on ds_{multi}, it can be known that the combined ds must have the "outstanding_1" pattern with the highlight of Feb, and there must not be "attribution", "evenness", "trend", and "seasonality" types. Therefore, the possible pattern types of ds after combination are only "outstanding_last", "change_point" and "outlier" (Step ①). Then, the system obtains ds_{multi}'s data by aggregating data from two ds_{single}'s (Step ②). Finally, the system calls the pattern analysis programs to calculate the three pattern types respectively. This step requires computing all (eight) pattern types without pruning inferences (Step ③).

Pattern snippets pruning for ds_{drill}. The calculation process of data and pattern of ds_{drill} is similar to the ds_{multi}. The drill-down process can be regarded as a process of further splitting the original subset (represented as ds_{raw}). Therefore, the inference of pattern snippet of ds_{drill} can also be pruned with ds_{raw}. Given the data scope $ds_{raw} = \{v_1, \ldots, v_n\}$, by expanding its breakdown item according to the drill-down attribute $drilldown$, we can get the data scope ds. The process

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

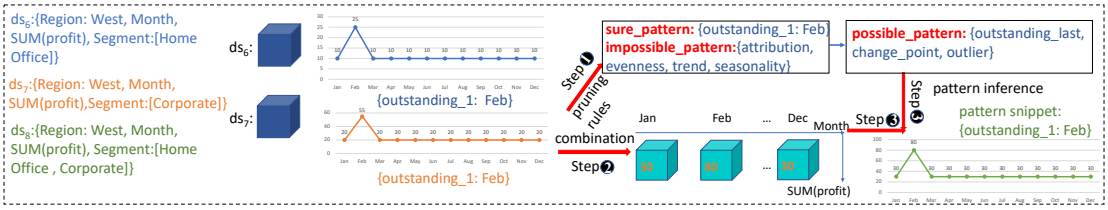


Fig. 5. Example of pattern inference process for ds_{multi} (ds_8) with pruning rules

of inferring the partial pattern snippet of ds from the pattern of ds_{raw} is: $Drill(ds_{raw}.ps) = \{sure_ps, impossible_ps\}$, where $Drill(\cdot)$ represents the constraint relationship between the patterns before and after drilling, $sure_ps$ and $impossible_ps$ represent the pattern types and highlights that must exist and pattern types that do not exist in ds_{multi}. At this time, the potential pattern types that ds need to infer through the pattern analysis programs are $possible_ps = type \setminus (sure_ps \bigcup impossible_sp)$. The drill-down process is also related to the aggregation, so the pruning rules are also analyzed from the perspective of four aggregations. We enumerate and analyze the pruning rules on ds_{drill} for four aggregations.

Property 2 (Pruning Rules for Pattern Inference on ds_{drill}): We find a total of 12 rules that can be used for pruning. Take a rule in the sum aggregation as an example. The rule is: if ds_{raw} have "outstanding_1" before drilling, then ds must have "outstanding_1" after drilling, and the highlight must be in the groups which are obtained by drilling down from the original highlight of the pattern type. This rule also applies to the "outstanding_n" pattern type.

Proof 2: Assume that the mean value of ds_{raw} is a, and each value is divided into m boxes during the drill-down process, that is, the drill-down ds is $\{v_1^{'}, \ldots, v_m^{'}, \ldots, v_{n*m}^{'}\}$. Without loss of generality, let v_1 in ds_{raw} before drilling be the highlight of the "outstanding_1" type, that is, $2*a < v_1 = v_1^{'} + \ldots + v_m^{'} < (n*a)/2$, let $v_k^{'} = max\{v_1^{'}, \ldots, v_m^{'}\}$, there is $m * v_k^{'} > \sum_{i=1}^{m} v_i^{i} > 2 * a$, so there is $v_k^{'} > 2 * a/m$, that is, there is $v_k^{'}$ that is greater than twice the mean after binning, and because $v_k^{'} < v_1 = v_1^{'} + \ldots + v_m^{'} < (n*a)/2$, so $v_k^{'}$ is less than half of the sum , satisfying the definition of "outstanding_1". In the same way, it is easy to prove that the "outstanding_n" type also satisfies this rule.

The mentioned rule can be succinctly represented as $(x) \rightarrow \{\{x\}, \{\}\}, x \in \{o_1, o_l\}$. Other rules in the SUM are $(x) \rightarrow \{\{\}, \{y\}\}, \{x, y\} = \{e, t\}; (\neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{a, e\}$. Rules in the MAX are $(o_1) \rightarrow \{\{o_1\}, \{\}\}; (x) \rightarrow \{\{\}, \{e\}\}, x \in \{o_l, a\}; (x) \rightarrow \{\{\}, \{y\}\}, \{x, y\} = \{e, t\}; (\neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{a, trend, e\}$. Rules in the MIN are $(o_l) \rightarrow \{\{o_l\}, \{\}\}$; $(x) \rightarrow \{\{\}, \{e\}\}, x \in \{o_1, o_l, a, trend\}; (\neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{trend, e\}$. There are two rules in the AVG. The first one is $(\neg x) \rightarrow \{\{\}, \{x\}\}, x \in \{trend, e\}$. The second one is if ds satisfy e, then it doesn't satisfy $o_1, o_l, a, cp, outlier, trend$.

Through the aforementioned InsightCube and pruning strategies, the calculation process of any data scope can be accelerated. Subsequently, the data scopes containing pattern snippets will be expanded into homogeneous data scopes, followed by calculating its homogeneous pattern snippets and finally summarizing them to obtain insights. As the calculation of data scopes is the most time-consuming in insight discovery, the efficiency of insight discovery is also optimized.

IV. Evaluation

A. Experimental setup

Experimental equipment. We implemented InsightCube based on Python. All experiments are conducted on a Linux machine with Intel Xeon Silver 4208 CPU and 64GB RAM.

Experimental comparison method. We use MetaInsight [14] as the baseline, which is the state-of-the-art insight discovery solution. In addition to comparing with the original MetaInsight, we also use the traditional OLAP calculation engine Kylin to build the data cube to accelerate MetaInsight (called MetaInsight-Kylin), and this method will also be used as a comparison solution.

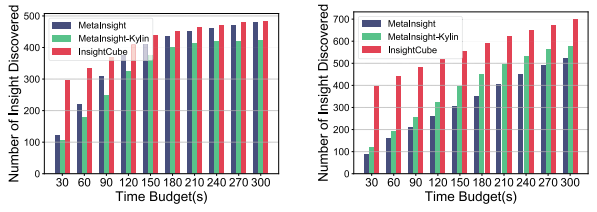
Experimental dataset. We select six real-world datasets collected from Kaggle for experiments: *superstore* (9,994 records with 10 attributes), *bike sales* (113,036 records with 9 attributes), *hotel booking* (119,390 records with 8 attributes), *house price* (168,446 records with 9 attributes), *disease* (308,854 records with 9 attributes), and *flights* (563,737 records with 8 attributes). We use and expand IDEBench [7] to generate meaningful workloads automatically.

Experiment evaluation metric. We use three metrics to evaluate the performance. 1) *Number of insights*: when the user's intention is a specified time budget or filtering operation, we use the number of insights calculated within a period of time as the metric to evaluate the efficiency of insight discovery. 2) *Time*: when the user's intention is the drill-down operation, the time of calculation on the same workload is used as the metric to evaluate the efficiency of insight discovery. 3) *Number of execution times*: for each dataset, we count the mean of the number of possible pattern types that exist in all workloads for each method as the number of execution times the method performed the pattern analysis programs.

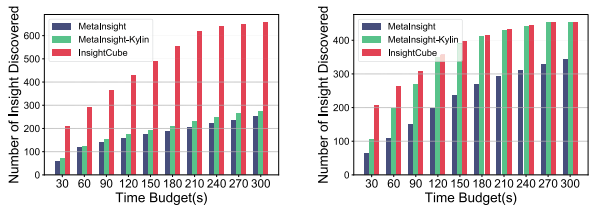
B. Experimental results of the method on datasets.

Number of insights for the filtering workload. The user's time budget intention can be regarded as a special filtering intention that is interested in the entire dataset, so when evaluating, merge it with the evaluation process of the filtering intention. For the filtering operation workload, we vary the

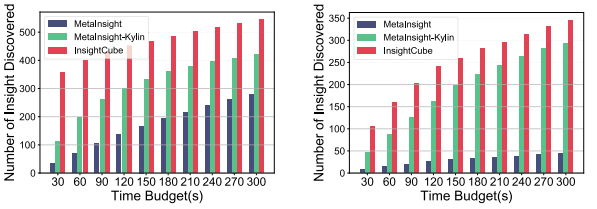
Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.



(a) superstore (b) bike sales

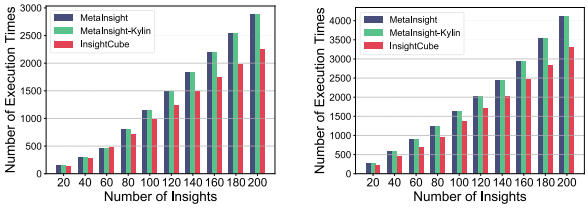


(c) hotel booking (d) house price

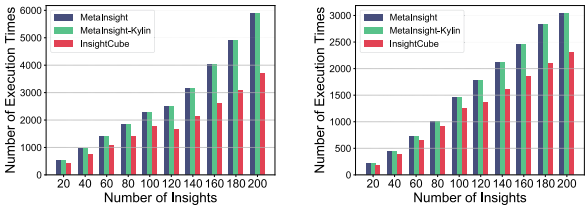


(e) disease (f) flights

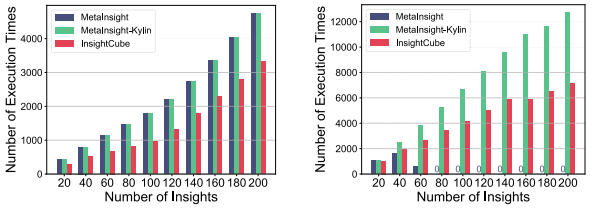
Fig. 6. The average efficiency of insight discovery for filtering operations



(a) superstore (b) bike sales



(c) hotel booking (d) house price



(e) disease (f) flights

Fig. 7. The number of execution times of pattern analysis program for filtering operations

TABLE I
INSIGHT DISCOVERY EFFICIENCY OF THE DRILL-DOWN OPERATION

Dataset	MetaInsight		MetaInsight-Kylin		InsightCube	
	TC	NT	TC	NT	TC	NT
superstore	24.9s	870	20.6s	870	16.6s	676
bike sales	17.4s	580	9.2s	580	8.2s	434
hotel booking	47.7s	2282	38.0s	2282	31.5s	1624
house price	21.4s	415	8.5s	415	6.6s	323
disease	45.8s	695	13.1s	695	11.3s	539
flights	114.6s	808	18.3s	808	12.9s	570

time budget from 30 seconds to 300 seconds. We calculate the average number of insights discovered by each method in each dataset's workload per time budget. Experiment results are shown in Fig. 6. As shown in Fig. 6, InsightCube achieves about 251% improvements over MetaInsight and 64% improvements over MetaInsight-Kylin in workloads corresponding to filtering intentions. MetaInsight-Kylin has obvious effects on large datasets but no obvious effect on small datasets. In addition to optimization in MetaInsight-Kylin, InsightCube also saves the pattern snippets in the offline stage and adopts the pruning strategies to accelerate the pattern inference process in the online stage.

Time of calculation for drill down workload. For drill-down workloads, the experiment calculated the sum of the calculating time overhead of the insight discovery process for different datasets. The time overhead of the three methods is shown in Table I, where TC represents the time cost of executing the drill-down workload, and the NT represents the number of times of calling pattern analysis programs in the drill-down process workload. It can be found from the table that when calculating the same number of insights, InsightCube's calculation time overhead is reduced by about 59% and 20% compared to MetaInsight and MetaInsight-Kylin, respectively. InsightCube uses not only the data cells but also the pattern cells and pruning strategies to reduce the number of times of calling the pattern analysis programs.

The number of execution times of the pattern analysis programs. We count the execution times of the pattern analysis programs in the online stage of insight discovery. Specifically, we use 300 seconds as the time budget to analyze the number of executions of the pattern analysis programs when different methods discover the same number of insights. We conduct experiments with different numbers of insights and perform pattern analysis on different datasets in the insight discovery process corresponding to filtering and drill-down operations. Experimental results are shown in Fig. 7 and Table I, respectively. It can be found that when calculating the same number of insights, the number of pattern analysis programs executed by InsightCube is reduced by about 20.7% compared with MetaInsight and MetaInsight-Kylin in the workload corresponding to filtering intentions. The workload corresponding to drill-down intentions is reduced by about 25.1%. This shows that the pruning strategies proposed in this paper can reuse the pattern snippets stored in the InsightCube and accelerate the pattern inference process of non-precomputed data scopes.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PRECOMPUTE THE TIME AND SPACE OVERHEAD OF THE INSIGHTCUBE

Dataset	Data Cell			Pattern Cell		
	Method	Time	Space	Method	Time	Space
superstore	Normal	25s	2.3MB	All	>24h	>100MB
	Augment	5s		Partial	3.99h	17.1MB
bike sales	Normal	131s	4.8MB	All	>24h	>100MB
	Augment	45s		Partial	16.6h	70.1MB
hotel booking	Normal	31s	1.1MB	All	>24h	>100MB
	Augment	8s		Partial	7.8h	15.6MB
house price	Normal	111s	0.2MB	All	>24h	>100MB
	Augment	36s		Partial	0.4h	2.1MB
disease	Normal	242s	2.4MB	All	>24h	>100MB
	Augment	79s		Partial	2.4h	10.6MB
flights	Normal	254s	1.8MB	All	>24h	>100MB
	Augment	70s		Partial	22.9h	21.3MB

MetaInsight and MetaInsight-Kylin have the same number of times because the latter only precalculates the data cells and does not calculate and store any pattern snippets. For the flights dataset, since MetaInsight cannot calculate the specified number of insights in some workloads, we denote the number of executions in this case as 0.

Time and space overhead in the offline stage. We compare the time and space cost of calculating data cells using augment queries (denoted as Augment) and ordinary queries (denoted as Normal), as well as the time and space cost of calculating partial (denoted as Partial) and all (represented as All) pattern cells. Results are shown in Table II. It can be found from the table that the precomputation method used in this paper has friendly time and space overhead in the offline stage. Especially in the calculation of data cells, the augment query method used in our method saves an average of 71.2% of time than the ordinary query method.

V. RELATED WORK

Exploratory Data Analysis (EDA). Exploratory data analysis is developed from data exploration [11]. Some exploratory data analysis tools [9], [12], [18] implement exploratory analysis functions based on GUI. As the dimensions continue to increase, the labor cost of exploratory analysis continues to increase. To solve this problem, some automatic exploratory data analysis tools [2], [6], [16] are proposed. For example, MetaInsight [14] summarizes and categorizes diverse patterns into a unified framework, thereby uncovering insights with richer information.

Interactive Data Analysis (IDA). In recent years, Interactive Data Analysis methods [3]–[5], [8], [13], [15], [17] are proposed to meet users' real-time needs further. However, compared with our method, all of these methods are simpler and contain less information in their definition of insights. For example, Drcao [15] uses preferences as soft constraints to measure the value of insights. The method of Cao [3] focuses on all tuples of interest based on a user-specified subset of tuples of interest. Our method follows MetaInsight [14], which is the latest and most informative definition in the industry.

VI. CONCLUSION

We propose an interactive insight discovery framework, InsightCube, to make the insight discovery process more efficient. In InsightCube, we precompute and store the data and pattern snippets of partial subsets of the dataset during the offline stage to avoid redundant calculations. Then, we propose pruning strategies to accelerate the pattern inference process for unknown subsets. Our experiments on multiple datasets demonstrate the effectiveness of InsightCube.

ACKNOWLEDGMENTS

This work was supported by National Natural Science Foundation of China No. 62072113.

REFERENCES

[1] Sihem Amer-Yahia, Tova Milo, and Brit Youngmann. Subdex: Exploring ratings in subjective databases. In *ICDE*, pages 2653–2656, 2021.
[2] Simon Brugman. pandas-profiling: exploratory data analysis for python. *https://github.com/pandas-profiling/pandas-profiling*, 2019.
[3] Yukun Cao, Xike Xie, and Kexin Huang. Learn to explore: on bootstrapping interactive data exploration with meta-learning. In *ICDE*, pages 1720–1733, 2023.
[4] Alexandre Chanson, Nicolas Labroche, Patrick Marcel, Stefano Rizzi, and Vincent T'kindt. Automatic generation of comparison notebooks for interactive data exploration. In *EDBT*, pages 2:274–2:284, 2022.
[5] Çagatay Demiralp, Peter J. Haas, Srinivasan Parthasarathy, and Tejaswini Pedapati. Foresight: Rapid data exploration through guideposts. *arXiv preprint arXiv:1709.10513*, 2017.
[6] Rui Ding, Shi Han, Yong Xu, Haidong Zhang, and Dongmei Zhang. Quickinsights: Quick and automatic discovery of insights from multi-dimensional data. In *SIGMOD*, pages 317–332, 2019.
[7] Philipp Eichmann, Emanuel Zgraggen, Carsten Binnig, and Tim Kraska. Idebench: A benchmark for interactive data exploration. In *SIGMOD*, pages 1555–1569, 2020.
[8] Magdalini Eirinaki, Suju Abraham, Neoklis Polyzotis, and Naushin Shaikh. Querie: Collaborative database exploration. *IEEE Trans. Knowl. Data Eng.*, 26(7):1778–1790, 2014.
[9] Ju Fan, Guoliang Li, and Lizhu Zhou. Interactive SQL query suggestion: Making databases user-friendly. In *ICDE*, pages 351–362, 2011.
[10] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub totals. *Data Min. Knowl. Discov.*, 1(1):29–53, 1997.
[11] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. Overview of data exploration techniques. In *SIGMOD*, pages 277–281, 2015.
[12] SAS Institute. Sas: Analytics, artificial intelligence, and data management. 2020.
[13] Doris Jung Lin Lee, Dixin Tang, Kunal Agarwal, Thyne Boonmark, Caitlyn Chen, Jake Kang, Ujjaini Mukhopadhyay, Jerry Song, Micah Yong, Marti A. Hearst, and Aditya G. Parameswaran. Lux: Always-on visualization recommendations for exploratory dataframe workflows. *Proc. VLDB Endow.*, 15(3):727–738, 2021.
[14] Pingchuan Ma, Rui Ding, Shi Han, and Dongmei Zhang. Metainsight: Automatic discovery of structured knowledge for exploratory data analysis. In *SIGMOD*, pages 1262–1274, 2021.
[15] Dominik Moritz, Chenglong Wang, Greg L. Nelson, Halden Lin, Adam M. Smith, Bill Howe, and Jeffrey Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco. *IEEE Trans. Vis. Comput. Graph.*, 25(1):438–448, 2019.
[16] Jinglin Peng, Weiyuan Wu, Brandon Lockhart, Song Bian, Jing Nathan Yan, Linghao Xu, Zhixuan Chi, Jeffrey M. Rzeszotarski, and Jiannan Wang. Dataprep.eda: Task-centric exploratory data analysis for statistical modeling in python. In *SIGMOD*, pages 2271–2280, 2021.
[17] Tarique Siddiqui, Albert Kim, John Lee, Karrie Karahalios, and Aditya G. Parameswaran. Effortless data exploration with zenvisage: An expressive and interactive visual analytics system. volume 10, pages 457–468, 2016.
[18] TIBCO Spotfire. Data visualization and analytics software, 2017.
[19] Bo Tang, Shi Han, Man Lung Yiu, Rui Ding, and Dongmei Zhang. Extracting top-k insights from multi-dimensional data. In *SIGMOD*, pages 1509–1524, 2017.
[20] John W Tukey et al. *Exploratory data analysis*, volume 2. Reading, MA, 1977.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:52:07 UTC from IEEE Xplore. Restrictions apply.